PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
BACHARELADO EM CIÊNCIAS DE DADOS E INTELIGÊNCIA ARTIFICIAL
PROGRAMAÇÃO E ESTRUTURA DE DADOS

Breno Martin de Toledo – RA 26011547
Claudinei Alves Pereira Reis – RA 26019325
Guilherme de França Rocha – RA 26011304
Mateus de Amorim Maiali – RA 26010939

MONITORAMENTO DE LABORATÓRIOS

1º Projeto (A1)

Profa. Lúcia F. A. Guimarães
Prof. Leonardo Rezende Costa

CAMPINAS
2026

1 INTRODUÇÃO

Este relatório apresenta o 1º Projeto (A1) da disciplina Programação e Estrutura de Dados: um programa em linguagem C que acompanha a ocupação dos laboratórios de informática de uma instituição de ensino ao longo de um período de dias e gera indicadores para apoiar a coordenação.

O programa registra a ocupação de até **20 laboratórios** durante até **30 dias**. Cada laboratório tem uma capacidade fixa de alunos e uma média de desempenho informada pelo usuário. A partir desses dados, o programa calcula a ocupação total de cada dia, a média e a taxa de ocupação de cada laboratório, os picos de ocupação e o dia de maior movimentação, e classifica os laboratórios combinando a taxa média de ocupação com a média de desempenho.

O **objetivo** do projeto é praticar as estruturas de dados vistas em sala (vetores e matrizes), os comandos de seleção e de repetição, a organização do código em funções e a validação de entradas, resolvendo um problema próximo do real: identificar horários de pico, laboratórios subutilizados e situações que pedem atenção da coordenação.

O relatório está organizado da seguinte forma: a seção 2 descreve o desenvolvimento, com a organização do programa, as estruturas usadas para guardar os dados, as validações, os cálculos, o critério de classificação, as dificuldades encontradas e os testes realizados; e a seção 3 traz as considerações finais.

2 DESENVOLVIMENTO

2.1 Organização do programa

Todo o código está em um único arquivo `.c`. O programa é dirigido por um menu principal que se repete (laço `do-while`) até o usuário escolher a opção 0 (Encerrar), e cada opção é tratada por um `switch`. O menu segue a numeração do enunciado: opções 1 a 6 e 0 para encerrar.

Quadro 1 – Opções do menu principal e funções correspondentes

Opção do menu	Função	O que faz
1 – Cadastrar dados	`menu_cadastro`, `cadastro_manual`, `cadastro_aleatorio`, `validar_dados`, `calcular_indicadores`	Lê os dados com validação, ou gera dados aleatórios para testes, e em seguida calcula os indicadores
2 – Exibir tabela de ocupação	`exibir_tabela_ocupacao`	Tabela laboratório × dia
3 – Calcular indicadores	`exibir_indicadores`, `exibir_maior_menor_ocupacao`	Dados inseridos, total diário, média diária, maior e menor ocupação, dia de maior movimentação e taxa média
4 – Laboratório mais ocupado	`laboratorio_mais_ocupado`	Laboratório(s) com maior ocupação em um dia informado
5 – Classificação dos laboratórios	`classificacao_laboratorios`, `obter_classificacao`	Classifica cada laboratório (seção 2.5)
6 – Exibir relatório	`menu_relatorio`, `relatorio_laboratorio`, `relatorio_final`, `imprimir_relatorio_lab`, `imprimir_dias_com_valor`	Relatório de um laboratório ou de todos
0 – Encerrar	—	Encerra o programa

Fonte: elaborado pelos autores.

Além delas, duas funções auxiliares são usadas em todo o programa: `limpar_buffer`, que descarta o resto da linha depois de cada `scanf` (seção 2.6), e `pausar`, que segura a tela até o usuário apertar ENTER. Ao todo, o programa tem 18 funções além da `main`.

O programa também impede que as opções sejam usadas fora de ordem. A variável de controle `ehtrue` registra se já houve cadastro: se o usuário pedir a tabela, os indicadores, a classificação ou o relatório antes de cadastrar, o programa avisa e volta ao menu. Os indicadores são calculados automaticamente logo após cada cadastro (função `calcular_indicadores`).

2.2 Estruturas usadas para guardar os dados

Os dados são guardados em **vetores** e em uma **matriz**, alocados de forma estática com o tamanho máximo permitido pelo enunciado. Os limites e as demais constantes foram definidos com `#define`.

O usuário informa quantos laboratórios (`num_labs`) e quantos dias (`qtd_dias`) quer acompanhar, e o programa usa apenas as primeiras posições de cada estrutura.

Quadro 2 – Estruturas de dados do programa

Variável	Tipo e dimensão	Conteúdo
`lab_day`	`int [LABS][DAYS]` (matriz 20 × 30)	Número de alunos de cada laboratório (linha) em cada dia (coluna)
`cap_labs`	`int [LABS]`	Capacidade de cada laboratório
`m_desemp`	`double [LABS]`	Média de desempenho de cada laboratório (0 a 10)
`total_dia`	`int [DAYS]`	Soma da ocupação de todos os laboratórios em cada dia
`media_diaria`	`float [LABS]`	Média diária de ocupação de cada laboratório
`taxa_media`	`float [LABS]`	Taxa média de ocupação (%) de cada laboratório
`num_labs`, `qtd_dias`	`int`	Quantidade de laboratórios e de dias efetivamente usados

Fonte: elaborado pelos autores.

Os vetores usam o mesmo índice da linha da matriz: a posição `i` de cada vetor se refere ao laboratório `i + 1`. Da mesma forma, `total_dia[j]` se refere à coluna `j` da matriz, ou seja, ao dia `j + 1`.

Como as funções precisam alterar o número de laboratórios e de dias lidos no cadastro, essas duas variáveis são passadas por referência (`int *num_labs`, `int *qtd_dias`). Vetores e matrizes já são passados por referência em C, então as funções de cadastro escrevem diretamente nas estruturas declaradas na `main`.

2.3 Validação das entradas

O enunciado exige impedir valores inválidos. O nosso time definiu as faixas válidas do Quadro 3. Toda leitura segue o mesmo padrão: ler o valor, descartar o resto da linha e repetir a pergunta enquanto o valor estiver fora da faixa.

Quadro 3 – Faixas válidas de cada entrada

Entrada	Faixa aceita	Motivo
Número de laboratórios	1 a 20	Limite do enunciado
Capacidade de cada laboratório	maior que 0	Evita divisão por zero no cálculo da taxa
Número de dias	1 a 30	Limite do enunciado
Ocupação de um laboratório em um dia	0 até a capacidade do laboratório	Um laboratório não comporta mais alunos que sua capacidade
Média de desempenho	0 a 10 (recusa também `nan`)	Escala de notas usada na universidade
Dia consultado (opção 4) e laboratório do relatório	1 até o número cadastrado	Evita acessar posições fora do que foi cadastrado
Opções dos menus	opções existentes	Qualquer outro valor, inclusive letras, mostra "Opcao invalida!"

Fonte: elaborado pelos autores.

A variável recebe um valor inválido (−1 ou 0) antes de cada `scanf`. Se o usuário digitar letras, o `scanf` não altera a variável, ela continua inválida e o laço pede o valor de novo. A função `limpar_buffer` descarta o que sobrou na linha.

Nos menus, a variável da opção começa em −1, e não em 0. Pois 0 significa "Encerrar" no menu principal e "Voltar" nos submenus.

Na média de desempenho, a condição do laço é escrita de forma negada: `!(m_desemp[i] >= 0 && m_desemp[i] <= 10)`. O `scanf("%lf")` aceita o texto `nan` (not a number), um valor que dá falso em qualquer comparação. Com a condição negada, ele também é recusado.

O cadastro aleatório segue as mesmas regras. Depois de sortear os dados, a função `validar_dados` confere cada valor com as faixas do Quadro 3. Se algum estiver fora, ela avisa qual foi e os dados são sorteados de novo.

2.4 Cálculo dos indicadores

Os indicadores são calculados percorrendo a matriz `lab_day` com dois laços `for` aninhados. Considerando L laboratórios e D dias:

- **Total diário de ocupação:** para cada dia, soma a coluna da matriz: total_dia[j] = Σ lab_day[i][j], com i de 0 a L − 1.
- **Média diária por laboratório:** para cada laboratório, soma a linha da matriz e divide pelo número de dias: media_diaria[i] = (Σ lab_day[i][j]) ÷ D.

- **Maior e menor ocupação:** o maior e o menor valor de toda a matriz, com o laboratório e o dia em que ocorreram. A matriz é percorrida duas vezes: a primeira passagem encontra os valores, e a segunda imprime todos os pares (laboratório, dia) iguais a eles, o que cobre os empates.
- **Dia de maior movimentação:** o maior valor em `total_dia`; todos os dias empatados são exibidos. O total desse dia é comparado com a capacidade total (soma de `cap_labs`), e o programa informa se ela foi atingida ou quantas vagas sobraram.
- **Taxa média de ocupação:** taxa_media[i] = media_diaria[i] ÷ cap_labs[i] × 100.
- **Laboratório mais ocupado em um dia:** o maior valor da coluna do dia informado pelo usuário; se houver empate, todos os laboratórios são listados.

As médias e as taxas usam variáveis `float`, e a média de desempenho usa `double`, para não perder as casas decimais. Isso evita o erro da divisão inteira.

2.5 Classificação dos laboratórios

O enunciado deixa o critério de classificação a cargo do nosso grupo, em conjunto com a professor, nas aulas de laboratório.

O critério adotado combina duas dimensões, cada uma dividida em três faixas:

- **Taxa média de ocupação:** baixa (menor que 50%), média (de 50% até menos de 80%) e alta (80% ou mais).
- **Média de desempenho:** baixo (menor que 5), médio (de 5 até menos de 8) e alto (8 ou mais).

O cruzamento das faixas gera a classificação do Quadro 4, implementada na função `obter_classificacao` com uma sequência de `if`/`else if`.

Quadro 4 – Critério de classificação (taxa de ocupação × desempenho)

Taxa média \ Desempenho	**Alto (≥ 8)**	**Médio (5 a < 8)**	**Baixo (< 5)**
Alta (≥ 80%)	Excelente	Muito bom	Atenção
Média (50% a < 80%)	Bom	Regular	Atenção
Baixa (< 50%)	Bom desempenho / baixa utilização	Subutilizado	Crítico

Fonte: elaborado pelos autores.

A lógica do critério é a seguinte. Um laboratório bem ocupado e com bom desempenho está sendo bem aproveitado (Excelente ou Muito bom). Um desempenho abaixo de 5 gera "Atenção" mesmo com boa ocupação, porque indica um problema pedagógico, e não de espaço. Com ocupação baixa, o laboratório é "Subutilizado": um sinal para a coordenação remanejar turmas. Ocupação e desempenho baixos ao mesmo tempo formam o caso "Crítico". A classificação aparece na opção 5 e nos relatórios individual e final.

Como a taxa é guardada em `float`, a comparação com as fronteiras de 50% e 80% usa uma pequena margem (`TOLERANCIA`), explicada na seção 2.6.

2.6 Dificuldades encontradas e soluções

Laço infinito com entrada não numérica. Quando o usuário digitava uma letra no lugar de um número, o `scanf("%d")` falhava e deixava a letra na entrada. O laço de validação lia a mesma letra de novo, para sempre. A solução foi criar a função `limpar_buffer`, chamada depois de todo `scanf`, que descarta os caracteres até o fim da linha. Ela também encerra o programa se a entrada terminar (EOF), o que evitou outro laço infinito ao rodar os testes com arquivos de entrada.

Cadastro aleatório gerando dados inválidos. Para testar sem digitar centenas de valores (20 × 30 = 600 ocupações), foi criado um cadastro aleatório com `rand()`. Para garantir que os dados sorteados seguem as mesmas regras do cadastro manual, foi acrescentada a validação descrita na seção 2.3.

Tabela larga demais para o terminal. Com 30 dias, a tabela de ocupação não cabia na largura da tela. A solução foi imprimir a tabela em blocos customizados (constante `BLOCO`), repetindo o cabeçalho a cada bloco.

Empates na maior e na menor ocupação. O enunciado exige mostrar todos os empates. Guardar apenas o primeiro maior valor encontrado não basta, porque outros pares (laboratório, dia) podem ter o mesmo valor. A solução foi percorrer a matriz duas vezes: a primeira passagem encontra o maior (e o menor) valor; a segunda imprime todos os pares cujo valor é igual ao encontrado. O time aplicou a mesma regra ao dia de maior movimentação, ao laboratório mais ocupado de um dia (opção 4) e à maior e à menor ocupação no relatório de cada laboratório: todos os empatados são listados.

Classificação errada na fronteira de 80%. O `float` nem sempre representa a conta exata. Com capacidade 9 e média de 7,2 alunos, a taxa é exatamente 80%, mas o `float` guarda 79,999992. A tela mostrava "80.00%" (o `printf` arredonda), enquanto o teste `taxa >= 80.0f` falhava e o laboratório caía na faixa média ("Bom" em vez de "Excelente"). Em testes automatizados testamos todas as combinações possíveis de capacidade (1 a 50), dias (1 a 30) e ocupação e encontramos 60 casos com esse erro, todos na fronteira de 80%. A solução foi somar uma margem de 0,001 (`TOLERANCIA`) à taxa antes de comparar. Ela não muda nenhum outro caso, porque duas taxas possíveis diferentes ficam a pelo menos 100 ÷ (50 × 30) ≈ 0,067 ponto percentual uma da outra.

Trabalho em equipe no mesmo arquivo. Como o código precisava ficar em um único `.c`, vários integrantes editavam o mesmo arquivo. O time usou um repositório no GitHub, com o GitHub Desktop, e escreveu um guia de uso para os integrantes.

2.7 Testes realizados

Os casos de teste foram escritos como arquivos de texto com uma entrada por linha, na ordem das perguntas do programa. Assim, cada caso pode ser repetido a cada nova versão, redirecionando o arquivo para o programa (`./labs < caso.txt`). Todo caso termina com a opção 0: se a saída acabar em "Fim da entrada" em vez de "Encerrando o programa", a sequência do arquivo saiu de ordem. O Quadro 5 resume os casos.

Quadro 5 – Casos de teste

Nº	O que testa	Resultado esperado e obtido
01	Exemplo do enunciado (4 laboratórios × 5 dias)	Indicadores iguais aos calculados à mão
02	Limite mínimo: 1 laboratório × 1 dia	Maior e menor ocupação na mesma célula; taxa de 100%; capacidade total atingida
03	Limite máximo: 20 laboratórios × 30 dias	Aceita os limites; tabela em blocos de 15 dias; 11 empates na menor ocupação
04	Empates na maior e na menor ocupação, no dia de maior movimentação e no laboratório mais ocupado do dia	Todos os empatados exibidos
05	Valores fora da faixa em todos os campos do cadastro (inclui `nan`)	Todos rejeitados, com nova pergunta
06	Capacidade 0 e negativa	Rejeitadas
07	Letras no menu, no submenu de cadastro e nos campos do cadastro	Rejeitadas, sem travar nem encerrar o programa
08	Opções do menu antes do cadastro e opções inexistentes	Aviso e retorno ao menu
09	Cadastro aleatório	Todos os valores dentro das faixas válidas
10	Dia de maior movimentação com todos os laboratórios lotados	"Atingiu a capacidade total!"
11	Taxa de exatamente 80% e de exatamente 50%	Classificação na faixa certa: Excelente (80%, nota 9) e Regular (50%, nota 6)

Fonte: elaborado pelos autores.

Todos os casos produziram o resultado esperado na versão final. Além deles, a classificação foi verificada de forma exaustiva: todas as combinações de capacidade, número de dias, ocupação total e notas nas fronteiras (3.566.250 casos) foram comparadas com a classificação calculada só com números inteiros, sem nenhuma divergência.

3 CONSIDERAÇÕES FINAIS

O programa atende às funcionalidades pedidas, com o menu na numeração do enunciado: cadastro com validação, tabela de ocupação, indicadores, consulta do laboratório mais ocupado em um dia, classificação e relatórios. O uso de uma matriz para a ocupação e de vetores paralelos para os dados de cada laboratório se mostrou simples e suficiente para os limites do problema.

As maiores dificuldades não estavam nos cálculos, mas na robustez da entrada de dados (entradas não numéricas, fim de entrada, valores fora da faixa), no tratamento de empates e na precisão do `float` nas fronteiras da classificação. Os casos de teste em arquivo foram decisivos para encontrar erros que passariam despercebidos em testes manuais. O projeto também exercitou o trabalho em equipe com controle de versão.